

PH3120
Computational
Physics Laboratory 1
CPL106 – Fourier
Transform Lab Sheet

1. I. $y(t) = A \sin(2\pi Ft + \phi)$

A – Amplitude: The peak deviation of the function from its center value.

F – Frequency: Number of complete cycles the sinewave completes per unit of time.

T – Time: Representing time.

ϕ - Phase Angle: Initial position of the waveform at $t=0$.

II. The relationship between the period of the sinewave (T) and its frequency (F) is an inverse one.

$$T = \frac{1}{F}$$

III.

$$T = \frac{1}{F}$$

IV. A higher sampling rate ensures more data points per cycle, leading to a more faithful digital representation and enabling accurate reconstruction of the original waveform. So, it is crucial in signal processing to accurately convert analog signals into digital ones.

V.

```
import numpy as np
import matplotlib.pyplot as plt

def sinewave(t, A, F, phi):
    return A*np.sin(2*np.pi*F*t+phi)

A = 0.1
F = 50
phi = np.pi/2

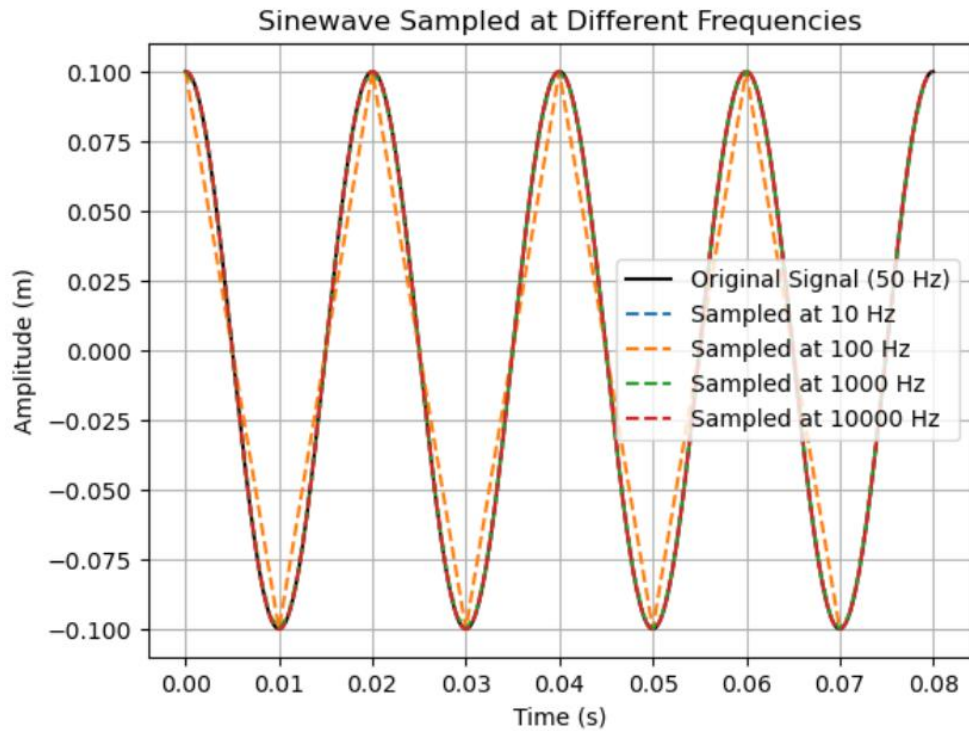
t_start = 0
t_end = 4 * (1 / F)

sampling_frequencies = [10, 100, 1000, 10000]

t_continuous = np.linspace(t_start, t_end, 500)
y_continuous = sinewave(t_continuous, A, F, phi)
plt.plot(t_continuous, y_continuous, label='Original Signal (50 Hz)', color='black',)

for fs in sampling_frequencies:
    t_sampled = np.arange(t_start, t_end, 1 / fs)
    y_sampled = sinewave(t_sampled, A, F, phi)
    plt.plot(t_sampled, y_sampled, label=f'Sampled at {fs} Hz', linestyle='--')

plt.title('Sinewave Sampled at Different Frequencies')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude (m)')
plt.legend()
plt.grid(True)
plt.show()
```



Sampling above the Nyquist rate (100 Hz, 1000 Hz, 10000 Hz) allows for accurate reconstruction. Higher sampling rates provide a more detailed and accurate digital representation of the analog signal, leading to better signal quality and simpler filter design for practical applications.

2. I. The time domain refers to the analysis of mathematical functions with respect to the time. When we describe a signal in the time domain, we are observing how its amplitude, or strength, changes over a period of time. Also, in Direct Fourier Transform transforms a signal from time domain into frequency domain.

II. The Frequency domain is a way of representing and analyzing signals based on the frequencies that compose them, rather than how their amplitude changes over time. When a signal is viewed in the frequency domain, the x-axis represents different frequencies, and the y-axis represents the amplitude and phase of each frequency component. The Fourier Transform is the mathematical tool that acts as the bridge between the time domain and the frequency domain.

III. The main difference between the Continuous Fourier Transform and the Discrete Fourier Transform depends on the nature of the signals they operate on and the nature of their outputs.

Continuous Fourier Transform (CFT)

Discrete Fourier Transform (DFT)

Input Signal	Continuous time, Continuous-amplitude signals.	Discrete time, Discrete-amplitude signals.
Output Spectrum	Produce a Continuous-frequency spectrum	Produce a Discrete-frequency spectrum
Nature	Analytical tool used for ideal continuous signals.	A computational tool, often implemented using the Fast Fourier Transform (FFT) algorithm, and is used for processing real-world sampled data.

(a) The Fourier Transform converts a discrete signal from the time domain to the frequency domain using the Discrete Fourier Transform (DFT).

Get Discrete-time signal $x[n]$ of length N,

$n = 0, 1, 2, \dots, N-1$

For DFT $X[k]$ denoted as

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j \frac{2\pi}{N} kn}$$

$X[k]$ represents the k^{th} frequency component in the discrete frequency domain.

And j is the imaginary unit ($j = \sqrt{-1}$).

- (b) The Fourier Transform converts a continuous signal from the time domain to the frequency domain using the Continuous Fourier Transform (CFT).

For a continuous-time signal $x(t)$,
Its CFT denoted as $X(\omega)$,

$$X(\omega) = \int_{-\infty}^{\infty} x(t)e^{-j\omega t} dt$$

$X(\omega)$ represents the frequency spectrum of the signal. ω is the continuous angular frequency in radians per second ($\omega=2\pi f$, where f is frequency in Hertz). j is the imaginary unit ($j = \sqrt{-1}$).

3. I. The Fast Fourier Transform (FFT) is an algorithm that computes the Discrete Fourier Transform (DFT) efficiently. It's not a different transform in itself, but rather a clever way to calculate the same result as the DFT, but significantly faster. Here the main advantages of FFT over DFT.

- Speed: This is the main advantage.
- Memory Efficiency: FFT algorithms are generally more memory – efficient than DFT.
- Enables Real-Time processing: Due to its speed, the FFT makes it possible to perform Fourier analysis on large datasets and in real-time applications where the direct DFT would be prohibitively slow.

II.

```
import numpy as np
import matplotlib.pyplot as plt

fs = 1000
t = np.linspace(0, 1, fs, endpoint=False)

f1, A1 = 100, 2
f2, A2 = 200, 1
sin1 = A1*np.sin(2*np.pi*f1*t)
sin2 = A2*np.sin(2*np.pi*f2*t)

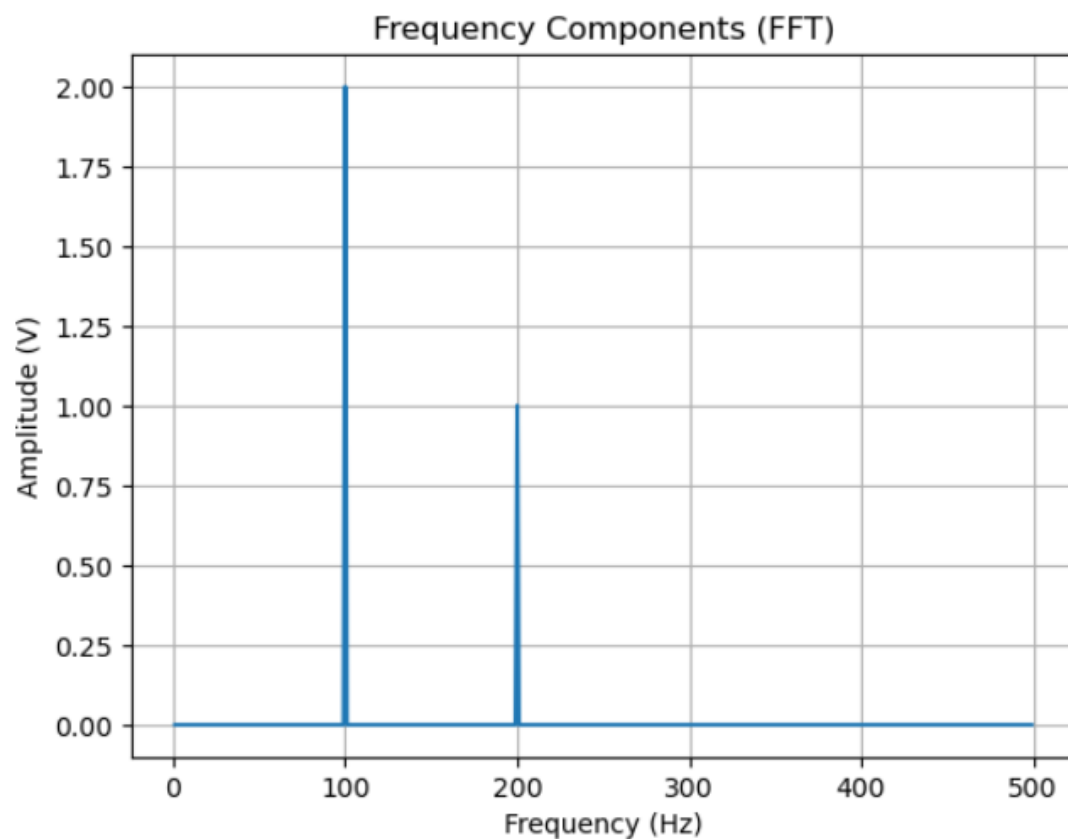
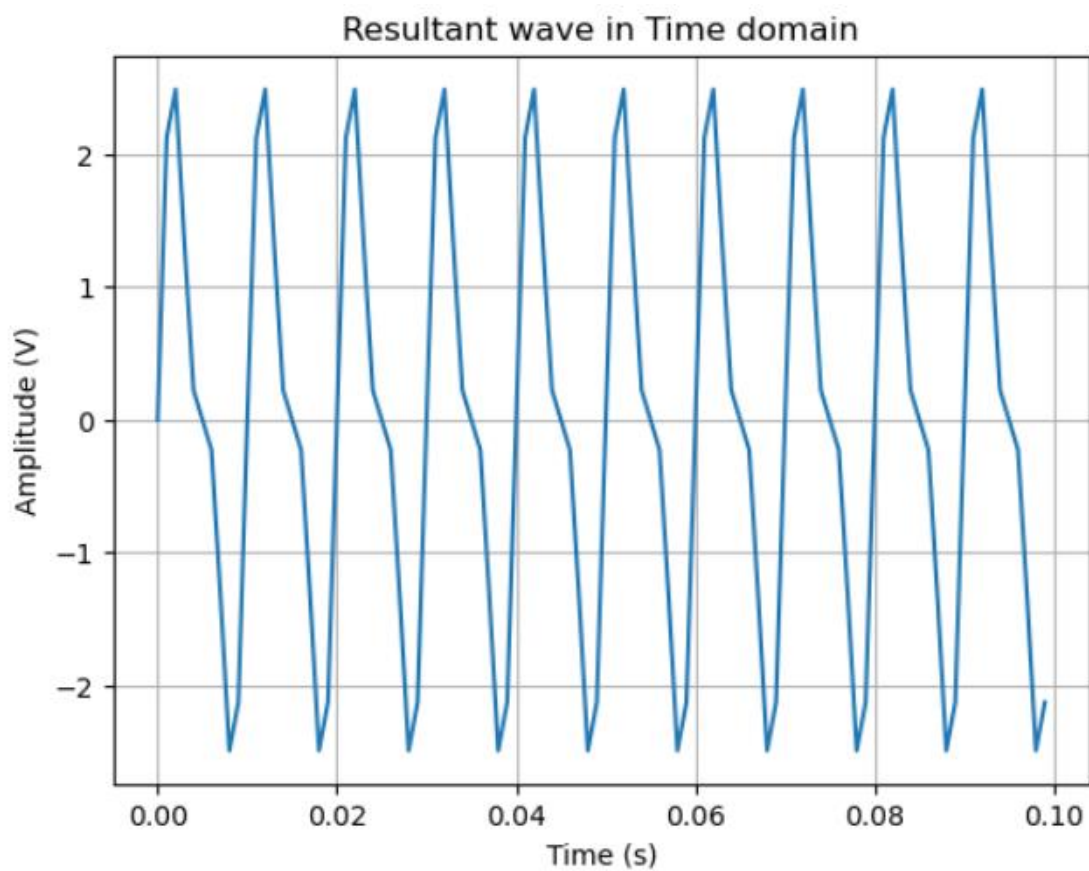
result = sin1 + sin2
fft_result = np.fft.fft(result)
freq = np.fft.fftfreq(len(fft_result), 1/fs)

positive_mask = freq>0
positive_freq = freq[positive_mask]
positive_magnitudes = np.abs(fft_result)[positive_mask]* 2/fs

plt.plot(t[:100], result[:100])
plt.title("Resultant wave in Time domain")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.grid(True)
plt.show()

plt.plot(positive_freq, positive_magnitudes)
plt.title('Frequency Components (FFT)')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude (V)')
plt.grid(True)
plt.show()

max_magnitudes = positive_magnitudes[positive_magnitudes > 0.1]
max_freq = positive_freq[positive_magnitudes > 0.1]
print("Detected Frequencies and Amplitudes:")
for freq, amp in zip(max_freq, max_magnitudes):
    print(f"Frequency: {freq:.1f} Hz, Amplitude: {amp:.2f} V")
```



Detected Frequencies and Amplitudes:
Frequency: 100.0 Hz, Amplitude: 2.00 V
Frequency: 200.0 Hz, Amplitude: 1.00 V

The FFT analysis reveals two distinct frequency components in the resultant wave.

A 100 Hz component with an amplitude of approximately 2 V, corresponding to the first sine wave and A 200 Hz component with an amplitude of approximately 1 V, corresponding to the second sine wave. These results confirm that the FFT accurately identifies the original frequencies and amplitudes of the superposed sine waves. The frequency resolution is sufficient due to the 1000 Hz sampling rate, which is well above the Nyquist frequency (400 Hz for the 200 Hz component).

III.

```
import numpy as np
import matplotlib.pyplot as plt

fs = 2000
t = np.linspace(0, 1, fs, endpoint=False)

#(a) Carrier wave
fc, Ac = 500, 5
sinc = Ac*np.sin(2*np.pi*fc*t)

#(b) Modulating Signal
fm, Am = 50, 2
sinm = Am*np.sin(2*np.pi*fm*t)

#(c) Modulated Signal
y = (Ac + sinm) * np.sin(2 * np.pi * fc * t)

#(d) Plot Vc(t), Vm(t) and y(t) for t in [0, 0.1]
t_plot = t[t <= 0.1]
plt.figure(figsize=(12, 10))

plt.subplot(3, 1, 1)
plt.plot(t_plot, sinc[t <= 0.1])
plt.title('Carrier Wave Vc(t)')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude (V)')

plt.subplot(3, 1, 2)
plt.plot(t_plot, sinm[t <= 0.1])
plt.title('Modulating Signal Vm(t)')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude (V)')

plt.subplot(3, 1, 3)
plt.plot(t_plot, y[t <= 0.1])
plt.title('Modulated Signal y(t)')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude (V)')
plt.show()
```



```

#(e)FFT algorithm
#(i) result plot
fft_result = np.fft.fft(y)
frequencies = np.fft.fftfreq(len(fft_result), 1/fs)
positive_mask = frequencies > 0
positive_frequencies = frequencies[positive_mask]
positive_magnitudes = np.abs(fft_result)[positive_mask] * 2 / fs

plt.plot(positive_frequencies, positive_magnitudes)
plt.title('Frequency Components of Modulated Signal')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude (V)')
plt.grid(True)

#(ii) Lowest frequency
threshold = 0.1
significant_indices = positive_magnitudes > threshold
for freq, amp in zip(positive_frequencies[significant_indices], positive_magnitudes[significant_indices]):
    plt.text(freq, amp, f'{freq:.0f} Hz', ha='center', va='bottom')
plt.tight_layout()
plt.show()

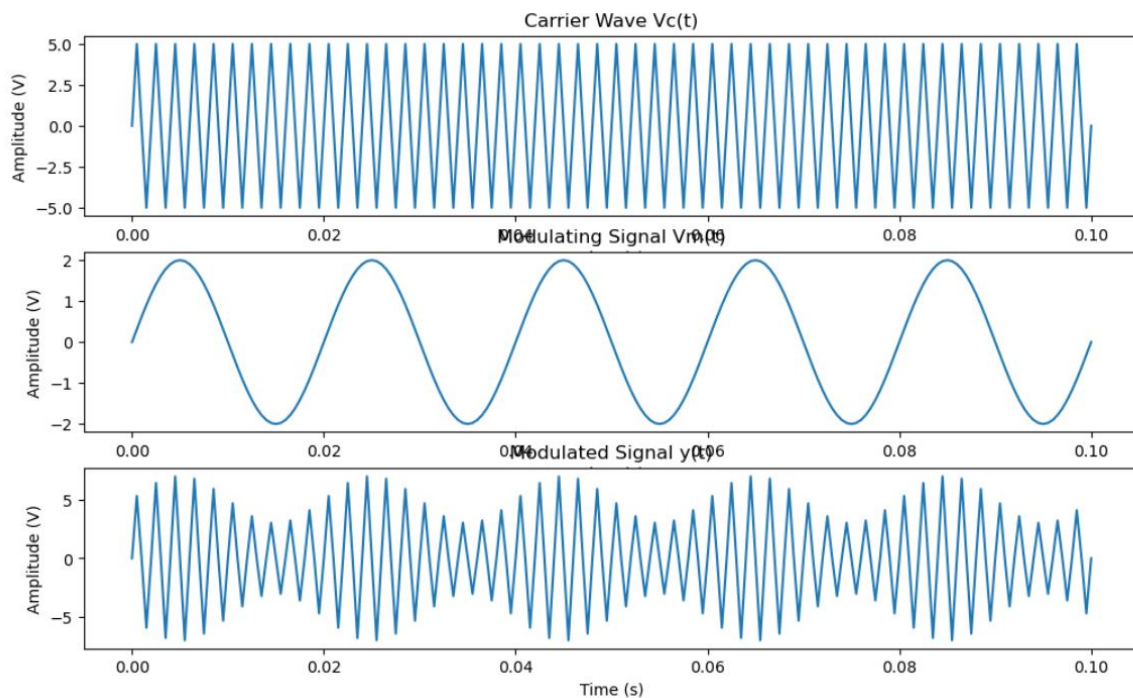
significant_freqs = positive_frequencies[significant_indices]
significant_mags = positive_magnitudes[significant_indices]
print("Detected Frequencies and Amplitudes:")
for freq, amp in zip(significant_freqs, significant_mags):
    print(f"Frequency: {freq:.1f} Hz, Amplitude: {amp:.2f} V")

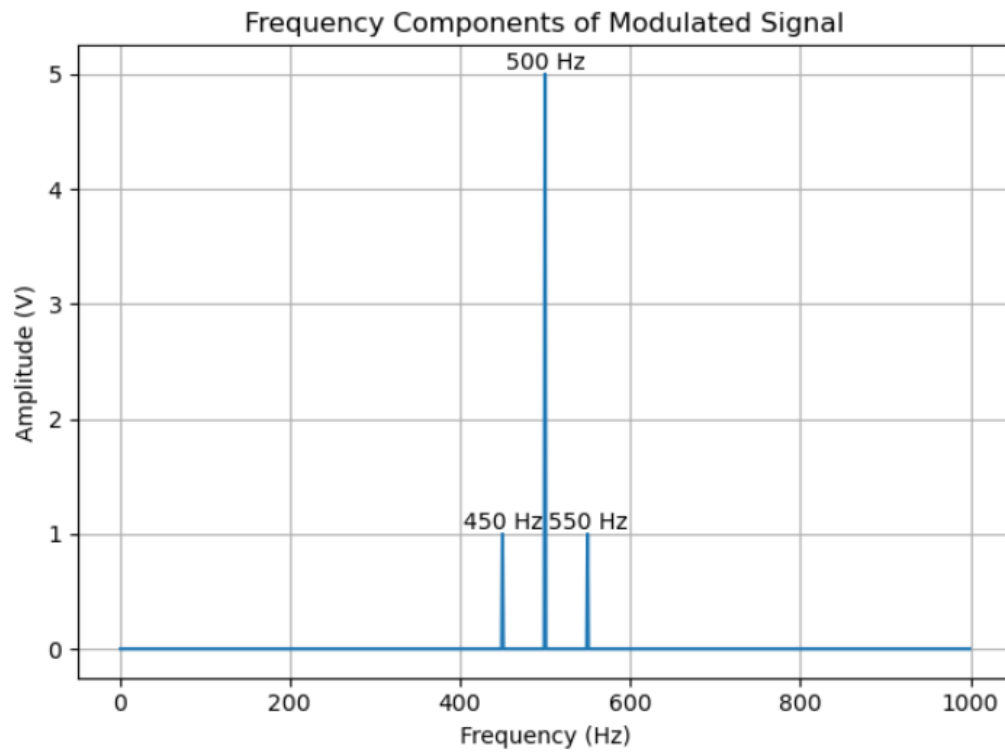
print(f"ii. Lowest frequency: {min(significant_freqs):.1f} Hz")

#(iii) Highest frequency
print(f"iii. Highest frequency: {max(significant_freqs):.1f} Hz")

#(iv) Band width based on the Fourier spectrum
print(f"iv. Bandwidth: {(max(significant_freqs) - min(significant_freqs)):.1f} Hz")

```





Detected Frequencies and Amplitudes:

Frequency: 450.0 Hz, Amplitude: 1.00 V

Frequency: 500.0 Hz, Amplitude: 5.00 V

Frequency: 550.0 Hz, Amplitude: 1.00 V

ii. Lowest frequency: 450.0 Hz

iii. Highest frequency: 550.0 Hz

iv. Bandwidth: 100.0 Hz

IV.

```
import numpy as np
import matplotlib.pyplot as plt

#(a) Function of modulated signal
fs = 2000
t = np.linspace(0, 1, fs, endpoint=False)

ym = 10*np.sin(200*np.pi*t) + 40*np.sin(100*np.pi*t)
yc = 20*np.sin(400*np.pi*t)

y = (20 + ym) * np.sin(400*np.pi*t)

#(b) Plot Vc(t), Vm(t) and y(t) for t in [0, 0.1]
t_plot = t[t <= 0.1]
plt.figure(figsize=(12, 7))

plt.subplot(3, 1, 1)
plt.plot(t_plot, yc[t <= 0.1])
plt.title('Carrier Wave Vc(t)')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude (V)')

plt.subplot(3, 1, 2)
plt.plot(t_plot, ym[t <= 0.1])
plt.title('Modulating Signal Vm(t)')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude (V)')

plt.subplot(3, 1, 3)
plt.plot(t_plot, y[t <= 0.1])
plt.title('Modulated Signal y(t)')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude (V)')
plt.show()
```

```

#(c)FFT algorithm
#(i) result plot
fft_result = np.fft.fft(y)
frequencies = np.fft.fftfreq(len(fft_result), 1/fs)
positive_mask = frequencies > 0
positive_frequencies = frequencies[positive_mask]
positive_magnitudes = np.abs(fft_result)[positive_mask] * 2 / fs

plt.plot(positive_frequencies, positive_magnitudes)
plt.title('Frequency Components of Modulated Signal')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude (V)')
plt.grid(True)

#(ii) Lowest frequency
threshold = 0.1
significant_indices = positive_magnitudes > threshold
for freq, amp in zip(positive_frequencies[significant_indices], positive_magnitudes[significant_indices]):
    plt.text(freq, amp, f'{freq:.0f} Hz', ha='center', va='bottom')
plt.tight_layout()
plt.show()

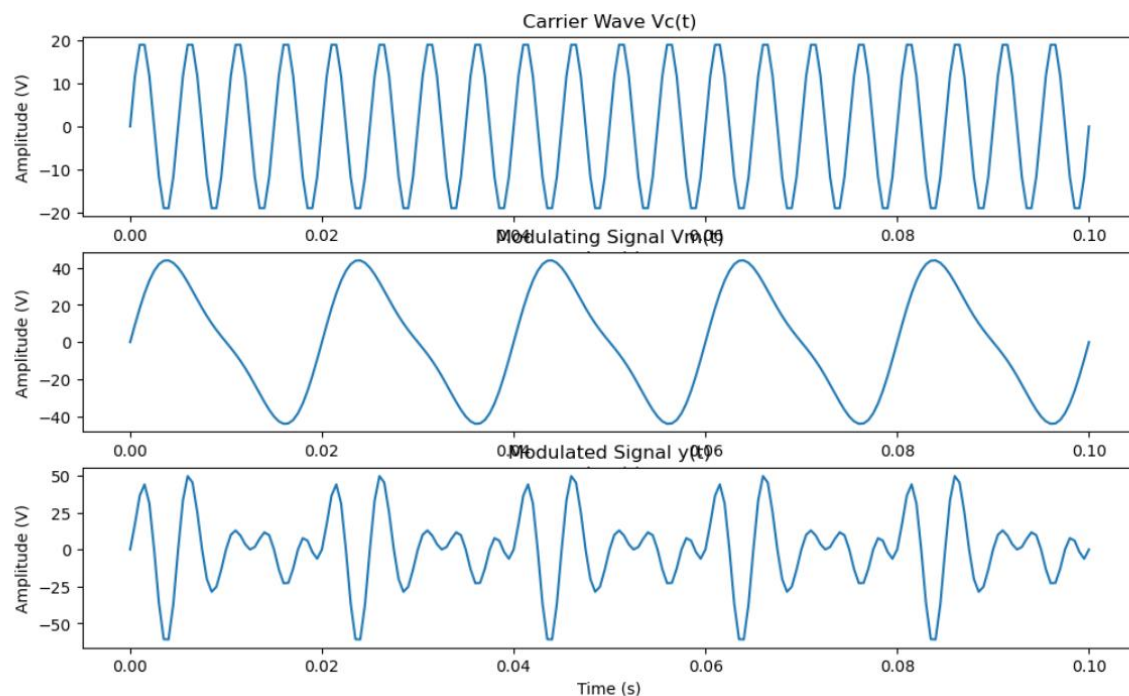
significant_freqs = positive_frequencies[significant_indices]
significant_mags = positive_magnitudes[significant_indices]
print("Detected Frequencies and Amplitudes:")
for freq, amp in zip(significant_freqs, significant_mags):
    print(f"Frequency: {freq:.1f} Hz, Amplitude: {amp:.2f} V")

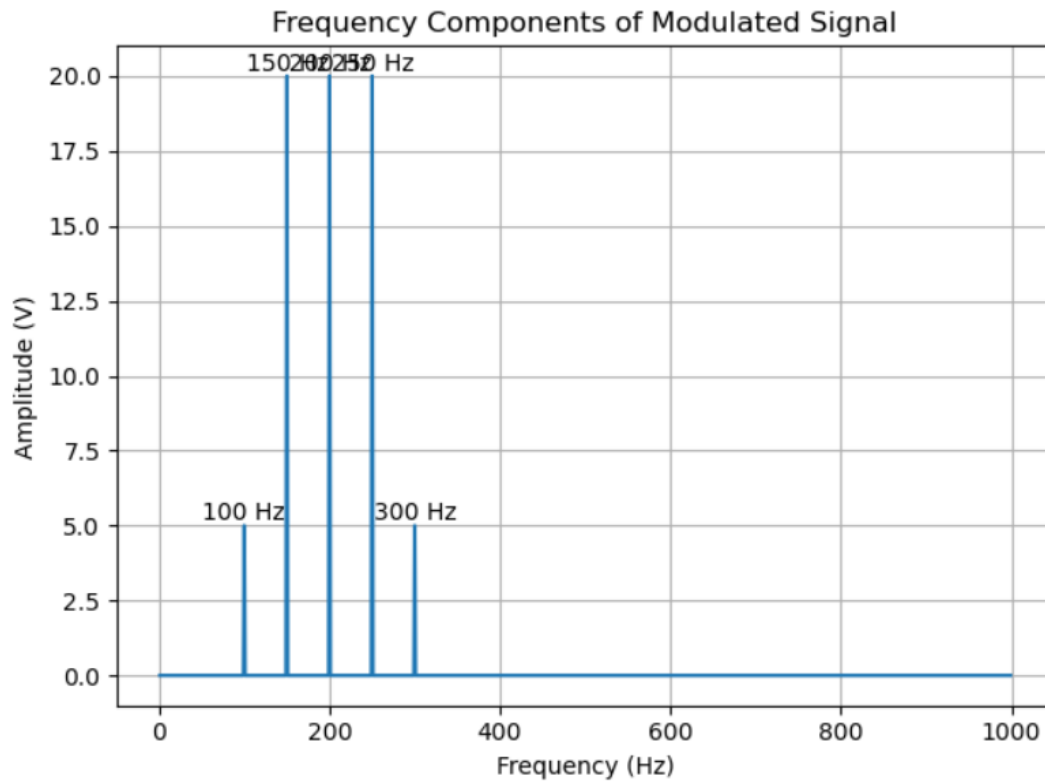
print(f"ii. Lowest frequency: {min(significant_freqs):.1f} Hz")

#(iii) Highest frequency
print(f"iii. Highest frequency: {max(significant_freqs):.1f} Hz")

#(iv) Band width based on the Fourier spectrum
print(f"iv. Bandwidth: {(max(significant_freqs) - min(significant_freqs)):.1f} Hz")

```





Detected Frequencies and Amplitudes:

Frequency: 100.0 Hz, Amplitude: 5.00 V

Frequency: 150.0 Hz, Amplitude: 20.00 V

Frequency: 200.0 Hz, Amplitude: 20.00 V

Frequency: 250.0 Hz, Amplitude: 20.00 V

Frequency: 300.0 Hz, Amplitude: 5.00 V

ii. Lowest frequency: 100.0 Hz

iii. Highest frequency: 300.0 Hz

iv. Bandwidth: 200.0 Hz